1. Introduction

1.1 What is Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) is a modern Natural Language Processing (NLP) technique that enhances the capabilities of large language models by combining information retrieval with text generation.

Traditional language models generate responses based only on their trained knowledge. However, they may produce incorrect or outdated answers if the required information is not available within the model. RAG solves this limitation by retrieving relevant information from external documents before generating responses.

In a RAG-based system, when a user submits a query, the system first searches a knowledge base to retrieve relevant content. This retrieved information is then passed to a language model, which uses it to generate accurate and context-aware responses.

The RAG process typically includes the following steps:

- Loading knowledge base documents
- Splitting documents into smaller chunks
- Generating embeddings for each chunk
- Storing embeddings in a vector database
- Retrieving relevant chunks based on user query
- Generating context-based responses
- This approach significantly improves response accuracy and reliability.

1.2 Why RAG is Needed

Modern customer support systems require fast, reliable, and accurate responses. Traditional rule-based chatbots often fail to understand complex user queries, while standalone language models may generate incorrect answers due to lack of updated knowledge.

RAG-based systems address these challenges by combining document retrieval with language generation. Instead of relying solely on stored knowledge inside the model, the system retrieves the most relevant information from real documents.

Key reasons for using RAG include:

- Improves response accuracy
- Provides context-based answers
- Reduces hallucination in language models
- Supports large document-based systems
- Enhances user trust and reliability

In customer support environments, RAG enables systems to deliver reliable solutions directly from official documentation.

1.3 Use Case Overview — Customer Support Assistant

The use case selected for this project is a Customer Support Assistant designed to answer user queries based on a predefined knowledge base.

Customer support departments handle a large number of repetitive queries related to common issues such as password resets, account access, service usage, and troubleshooting. Handling these requests manually can increase workload and response time.

This system automates customer support interactions by retrieving relevant information from a knowledge base document and generating meaningful responses.

The system supports the following features:

- Processing customer queries
- Retrieving relevant information from documents
- Generating contextual responses
- Detecting low-confidence answers
- Escalating complex queries to human support
- Delivering accurate customer responses

This design improves efficiency while maintaining response quality.

1.4 Project Scope

The scope of this project includes designing and implementing a Retrieval-Augmented Generation system capable of answering customer queries using document-based knowledge.

The system focuses on:

- Processing a customer support PDF document
- Chunking document content into smaller segments
- Generating embeddings using embedding models
- Storing embeddings in a vector database
- Retrieving relevant chunks based on similarity
- Generating responses using a language model
- Managing workflow using LangGraph

Supporting Human-in-the-Loop escalation

The project demonstrates how modern NLP systems integrate retrieval mechanisms with workflow orchestration to build intelligent customer support solutions.

---

2. System Architecture Explanation

2.1 Overview of System Architecture

The RAG-Based Customer Support Assistant system is designed using a modular architecture that integrates document processing, embedding generation, vector retrieval, language model response generation, and workflow orchestration.

The system processes customer queries by retrieving relevant information from a knowledge base document and generating responses using a language model. Each component performs a specific task to ensure efficient processing and accurate responses.

The overall architecture consists of multiple layers, including:

- User Interface Layer
- Document Processing Layer
- Embedding Layer
- Vector Storage Layer
- Retrieval Layer
- Language Model Layer
- Workflow Orchestration Layer
- Human-in-the-Loop (HITL) Layer

These layers work together to process queries and generate responses in a structured and reliable manner.

2.2 User Interface Layer

The User Interface Layer is responsible for receiving input queries from users and displaying responses.

In this system, the interface can be implemented using a Command Line Interface (CLI) or a simple web-based interface. Users enter queries in natural language format, and the system processes these queries through the backend workflow.

Responsibilities:

- Accept user queries
- Send queries to the processing module
- Display generated responses
- Notify users if escalation occurs

This layer ensures smooth interaction between users and the system.

2.3 Document Processing Layer

The Document Processing Layer is responsible for loading and preparing the knowledge base document.

The system reads the input PDF file and extracts text content. The extracted text is then divided into smaller segments using a chunking strategy.

Responsibilities:

- Load PDF document
- Extract text content
- Split text into chunks
- Prepare data for embedding generation
- Chunking improves processing efficiency and retrieval accuracy.

2.4 Embedding Layer

The Embedding Layer converts text chunks into numerical vector representations.

Each chunk is transformed into an embedding using a pre-trained embedding model. These embeddings capture semantic meaning, allowing the system to understand relationships between words and sentences.

Responsibilities:

- Convert text chunks into embeddings
- Store embedding vectors
- Maintain semantic relationships
- Embedding generation enables efficient similarity-based search.

2.5 Vector Storage Layer

The Vector Storage Layer stores embeddings generated from document chunks.

In this system, ChromaDB is used as the vector database. It allows efficient storage and retrieval of embedding vectors.

Responsibilities:

Store embedding vectors

Maintain mapping between chunks and embeddings

Enable fast similarity search

This layer supports scalable data storage.

2.6 Retrieval Layer

The Retrieval Layer identifies relevant document chunks based on user queries.

When a query is received, the system generates a query embedding and compares it with stored embeddings in the vector database. The most similar chunks are retrieved and passed to the language model.

Responsibilities:

- Generate query embeddings
- Perform similarity search
- Retrieve relevant chunks
- Send retrieved context to language model

This layer ensures context-based response generation.

2.7 Language Model Layer

The Language Model Layer generates responses using retrieved context.

A large language model processes the user query along with the retrieved document content. The model generates natural language responses based on the available information.

Responsibilities:

- Process retrieved context
- Generate responses
- Ensure meaningful output

This layer is responsible for producing user-readable responses.

2.8 Workflow Orchestration Layer (LangGraph)

The Workflow Orchestration Layer controls the sequence of operations using LangGraph.

LangGraph manages the flow between nodes such as input processing, retrieval, response generation, evaluation, and output delivery.

Responsibilities:

- Control execution flow
- Manage workflow states
- Perform routing decisions
- Handle escalation logic

This layer ensures structured query processing.

2.9 Human-in-the-Loop (HITL) Layer

The Human-in-the-Loop (HITL) Layer enables human intervention when automated responses are insufficient.

If the system detects low confidence or missing information, the query is escalated to a human operator.

Responsibilities:

- Receive escalated queries
- Provide manual responses
- Deliver accurate solutions
- Improve system reliability

HITL improves response quality and user trust.

---

3. Design Decisions

3.1 Chunk Size Selection

Chunk size plays an important role in the performance of a Retrieval-Augmented Generation (RAG) system. Chunking refers to dividing large documents into smaller sections before generating embeddings.

In this system, document text is divided into moderate-sized chunks to balance context quality and retrieval speed. If chunks are too large, retrieval becomes slower and less accurate. If chunks are too small, important contextual meaning may be lost.

A balanced chunk size improves the ability of the system to retrieve meaningful information while maintaining efficient performance.

Reasons for Selecting Moderate Chunk Size:

- Preserves meaningful context
- Improves retrieval accuracy
- Reduces irrelevant results
- Maintains efficient processing time

This decision helps optimize overall system performance.

3.2 Embedding Strategy Selection

Embedding strategy determines how text content is converted into numerical vector representations.

In this system, pre-trained embedding models are used to generate semantic representations of document chunks. These embeddings capture the meaning of words and phrases, allowing the system to perform similarity-based retrieval.

Using pre-trained embeddings eliminates the need to train embedding models from scratch, which saves computational resources and development time.

Reasons for Selecting Pre-trained Embeddings:

- Faster implementation
- High semantic understanding

- Reduced training complexity
- Efficient similarity matching

This decision improves the accuracy of document retrieval.

3.3 Vector Database Selection — ChromaDB

The vector database stores embeddings and supports similarity-based search.

ChromaDB was selected as the vector database because it provides efficient storage and retrieval capabilities. It is lightweight and easy to integrate into RAG-based workflows.

ChromaDB supports fast similarity searches and allows the system to scale efficiently when handling larger document sets.

Reasons for Selecting ChromaDB:

- Easy integration
- Fast vector search
- Lightweight architecture
- Supports scalable storage
- Suitable for document retrieval systems

This decision supports reliable embedding storage and retrieval.

3.4 Retrieval Approach Selection

The retrieval approach determines how relevant document chunks are selected.

In this system, similarity-based retrieval is used. The query embedding is compared with stored document embeddings, and the most similar chunks are selected.

This approach ensures that responses are generated using relevant information.

Reasons for Selecting Similarity-Based Retrieval:

- Provides context-aware retrieval
- Improves response accuracy

- Reduces irrelevant results
- Supports semantic search

This retrieval method enhances system effectiveness.

3.5 Prompt Design Strategy

Prompt design defines how instructions are provided to the language model.

In this system, prompts are structured to include both user queries and retrieved document context. This helps the language model generate accurate and meaningful responses.

Structured prompts reduce ambiguity and improve response clarity.

Reasons for Structured Prompt Design:

- Improves response accuracy
- Provides clear context
- Reduces irrelevant responses
- Enhances output quality

This design ensures better response generation.

3.6 Language Model Selection

The language model is responsible for generating responses based on retrieved information.

A transformer-based language model is selected because of its strong capability to understand context and generate natural language responses.

Transformer-based models provide better performance than traditional rule-based systems.

Reasons for Selecting a Transformer-Based Model:

- Strong contextual understanding
- High response quality
- Suitable for NLP tasks
- Supports large-scale processing

This decision improves the intelligence of the system.

3.7 Workflow Engine Selection — LangGraph

LangGraph is used as the workflow orchestration engine to manage processing steps.

LangGraph supports node-based workflow design, allowing structured execution flow between different components.

It enables conditional routing and supports Human-in-the-Loop (HITL) integration.

Reasons for Selecting LangGraph:

- Supports graph-based workflows
- Enables conditional routing
- Simplifies workflow management
- Supports HITL integration

This decision improves workflow control and flexibility.

---

4. Workflow Explanation

4.1 Overview of Workflow Execution

The workflow in the RAG-Based Customer Support Assistant is designed using a structured node-based architecture. Each step in the workflow performs a specific function, ensuring smooth processing of user queries.

LangGraph is used as the workflow orchestration engine to manage execution flow between nodes. It controls the sequence of operations and ensures that data moves correctly from one stage to another.

The workflow processes user queries in multiple stages, including input processing, retrieval, response generation, evaluation, and output delivery.

This structured workflow improves reliability, maintainability, and scalability.

4.2 Node Responsibilities

Each node in the workflow performs a dedicated function.

Input Processing Node

The Input Processing Node receives the user query and prepares it for processing.

Responsibilities:

- Accept user query
- Clean input text
- Convert the query into an embedding
- Pass the query to the retrieval module

This node ensures that user input is correctly prepared before processing.

Retrieval Node

The Retrieval Node searches the vector database to find relevant document chunks.

Responsibilities:

- Generate query embedding
- Search vector database
- Retrieve relevant chunks
- Send retrieved context to response generation node

This node ensures that the language model receives accurate context.

Response Generation Node

The Response Generation Node generates answers using retrieved information.

Responsibilities:

- Combine query with retrieved context

- Send prompt to language model
- Generate response text
- Pass response to evaluation node

This node produces the system's main response.

Evaluation Node

The Evaluation Node checks the reliability of generated responses.

Responsibilities:

- Evaluate response quality
- Measure confidence level
- Detect missing information
- Decide whether escalation is required

This node ensures that unreliable responses are not delivered.

Output Node

The Output Node delivers responses to users.

Responsibilities:

- Display generated response
- Notify escalation if triggered
- Record output logs

This node completes the response cycle.

4.3 Workflow State Transitions

Workflow state transitions define how information moves between nodes during execution.

A shared state object is used to store data that flows through the workflow.

State Variables Include:

user_query — Stores user input

retrieved_chunks — Stores retrieved context

generated_response — Stores model output

confidence_score — Stores evaluation score

escalation_flag — Indicates whether HITL is required

Each node reads and updates the state during execution.

This design ensures smooth coordination between workflow components.

4.4 Workflow Execution Sequence

The workflow follows a sequential execution pattern.

Execution Steps:

1. User enters query
2. Query moves to the Input Processing Node
3. Retrieval Node searches vector database
4. Relevant chunks are retrieved
5. Response Generation Node generates answer
6. Evaluation Node checks response confidence
7. If confidence is high → Output Node
8. If confidence is low → HITL escalation

This structured execution ensures accurate and reliable responses.

4.5 Advantages of Workflow-Based Design

Using a workflow-based system provides several benefits.

Advantages Include:

- Organized execution flow
- Easy debugging and maintenance
- Supports conditional routing
- Improves response reliability
- Enables HITL integration

This design improves system efficiency and flexibility.

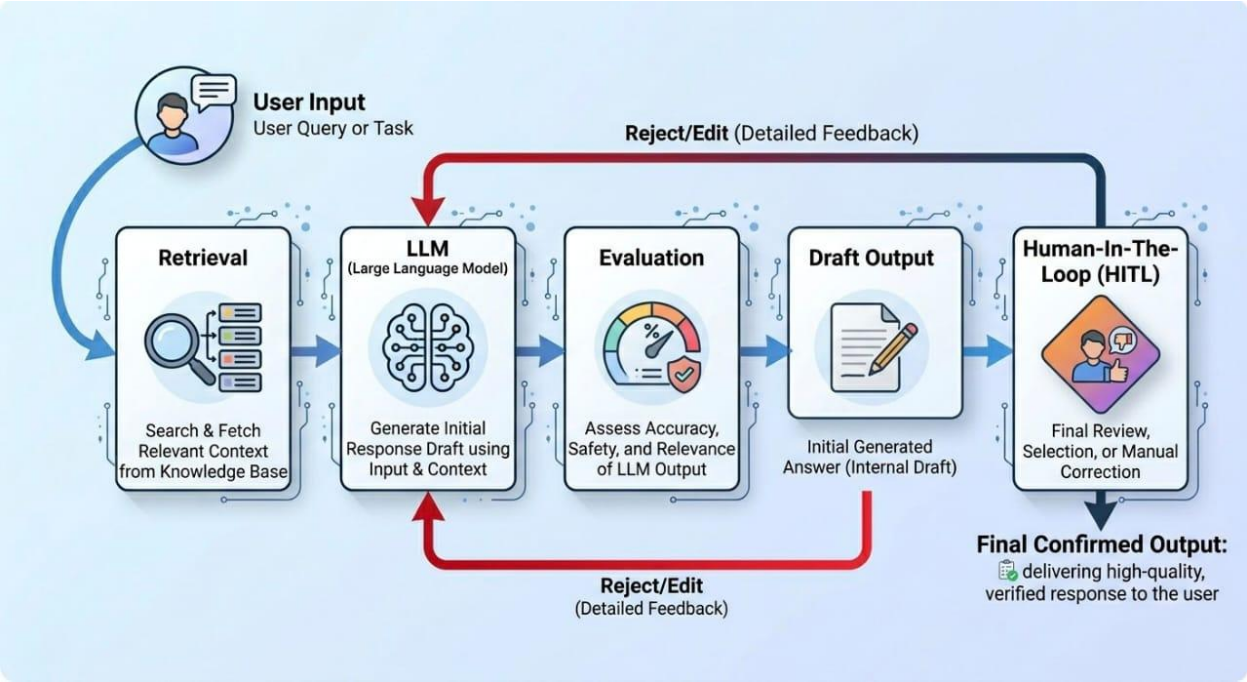

Fig. 1: Workflow of Langgraph

5. Conditional Logic

5.1 Overview of Conditional Logic

Conditional logic plays a crucial role in controlling the behavior of the RAG-Based Customer Support Assistant system. It determines how the system reacts to different scenarios during query processing.

The system evaluates responses generated by the language model and decides whether the response is reliable or requires escalation. This decision-making process ensures that users receive accurate and trustworthy answers.

Conditional logic improves the intelligence of the system by allowing dynamic routing based on response quality and retrieval results.

5.2 Intent Detection

Intent detection is used to understand the purpose of the user's query. It helps the system categorize queries and determine the appropriate response strategy.

In this system, intent detection is performed by analyzing the user query and identifying whether it matches the available knowledge base content.

Examples of User Intent:

- Request for account support
- Password reset assistance
- Service-related inquiries
- Troubleshooting questions

Understanding user intent allows the system to retrieve the most relevant information.

5.3 Routing Decision Logic

Routing decisions determine how the system proceeds after generating a response.

After retrieval and response generation, the system evaluates whether the generated response meets quality requirements.

Routing Decisions Include:

- If relevant chunks are found → Generate response
- If response is valid → Send to output
- If response quality is low → Trigger escalation
- If no chunks are found → Escalate immediately

This structured decision logic ensures proper workflow management.

5.4 Confidence-Based Routing

Confidence-based routing determines whether the generated response is reliable.

The system calculates a confidence score based on similarity between the query and retrieved document chunks.

Decision Conditions:

- Confidence Score ≥ Threshold → Output Response
- Confidence Score < Threshold → Escalate to HITL

Example Threshold:

Confidence Threshold = 0.60

If the confidence score falls below this threshold, the system avoids delivering unreliable responses.

5.5 Missing Context Detection

Missing context occurs when the system cannot find relevant document chunks that match the user query.

This situation indicates that the requested information is not available in the knowledge base.

Detection Conditions:

- No matching chunks found
- Retrieved chunks unrelated to query
- Empty retrieval results

When missing context is detected, the system automatically escalates the query to human support.

5.6 Escalation Trigger Conditions

Escalation is triggered when the system determines that automated processing is insufficient.

Escalation Conditions Include:

- Low confidence response
- Missing document context
- Complex or ambiguous query
- Incomplete generated response

Escalation prevents incorrect information delivery.

5.7 Benefits of Conditional Logic

Conditional logic improves system efficiency and reliability.

Key Benefits:

- Prevents incorrect responses
- Improves decision-making
- Supports intelligent routing
- Enhances system accuracy
- Enables reliable escalation

This logic plays a critical role in maintaining system trustworthiness.

---

6. HITL Implementation

6.1 Overview of Human-in-the-Loop (HITL)

Human-in-the-Loop (HITL) is an important feature of the RAG-Based Customer Support Assistant system. It allows human intervention when the automated system is unable to generate reliable responses.

Although automated systems provide fast responses, certain queries may be too complex or lack sufficient context. In such situations, human support ensures accurate and meaningful responses.

HITL enhances system reliability by combining machine intelligence with human judgment.

6.2 Role of Human Intervention

Human intervention plays a key role in maintaining response quality.

When the system identifies uncertain or incomplete responses, it forwards the query to a human support agent. The human agent reviews the query and provides an accurate response based on their understanding.

Responsibilities of Human Support:

- Review escalated queries
- Understand user request
- Provide accurate response
- Correct incomplete or unclear answers
- Ensure user satisfaction

Human involvement ensures that difficult queries are handled properly.

6.3 When Human Intervention is Triggered

Human intervention is triggered when the system detects conditions that indicate unreliable responses.

Trigger Conditions Include:

- Low confidence score detected
- No relevant chunks retrieved
- Retrieved content is incomplete
- Complex or ambiguous query identified
- Language model fails to generate a response

These conditions ensure that automated errors are minimized.

6.4 HITL Workflow Execution

The HITL workflow defines how escalated queries are handled.

Workflow Steps:

1. System processes user query
2. Response confidence is evaluated
3. If confidence is low, escalation is triggered
4. Query is forwarded to a human operator
5. Human reviews query and generates response
6. Human response is returned to the system

7. Response is delivered to the user

This structured workflow maintains smooth communication between the system and human support.

6.5 Integration of Human Response

After the human operator generates a response, the system integrates the response into the workflow.

Integration Steps:

- Receive response from human operator
- Store response in system logs
- Send final response to user
- Mark escalation as completed

This integration ensures consistent output handling.

6.6 Benefits of HITL Implementation

HITL improves system performance in several ways.

Key Benefits:

- Improves response accuracy
- Handles complex queries
- Prevents incorrect answers
- Enhances user trust
- Supports continuous system improvement

Human feedback can also be used to improve future system performance.

6.7 Limitations of HITL Implementation

Although HITL provides significant advantages, it introduces certain limitations.

Key Limitations:

- Requires human availability

- May increase response time
- Introduces additional operational effort
- Depends on human expertise

Despite these limitations, HITL significantly enhances system reliability.

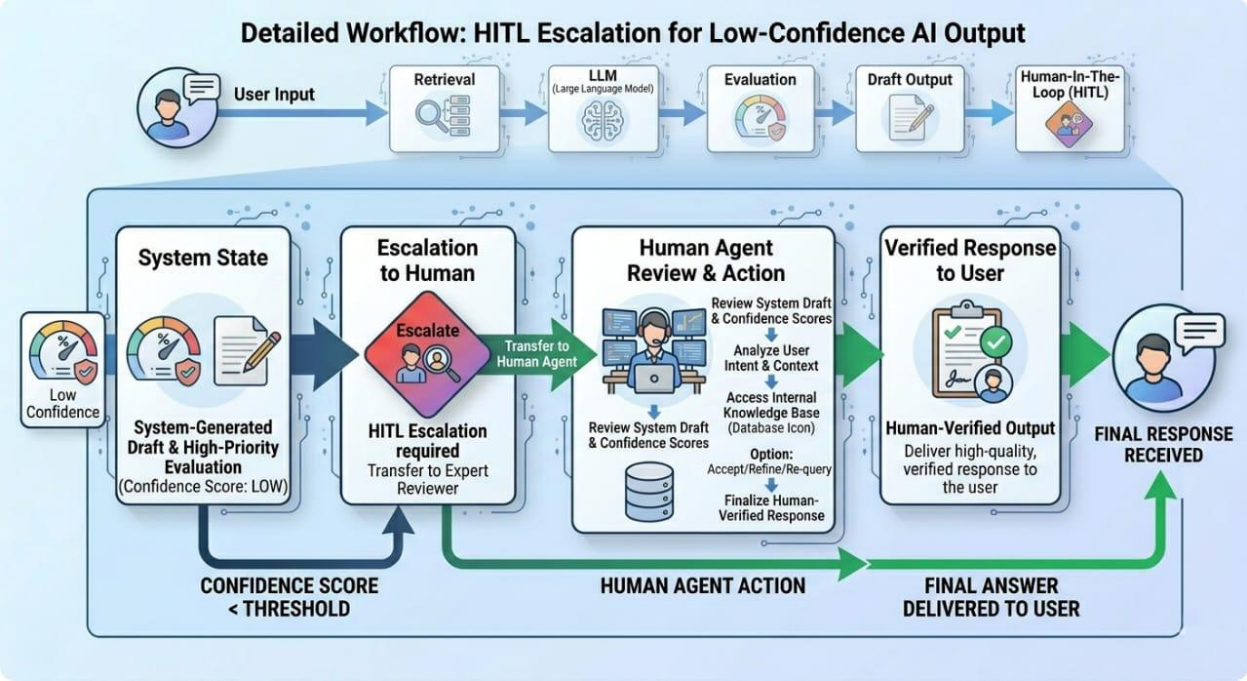

Fig 2: Workflow Diagram customer support

---

7. Challenges & Trade-offs

7.1 Overview of Challenges

Designing a RAG-Based Customer Support Assistant involves multiple technical challenges related to data processing, retrieval accuracy, response generation, and system performance.

These challenges arise because the system must balance speed, accuracy, and scalability while handling large volumes of data. Understanding these challenges helps in designing better and more efficient systems.

This section discusses key challenges encountered during the design of the system and the trade-offs made to address them.

7.2 Retrieval Accuracy vs Processing Speed

One major challenge in RAG systems is balancing retrieval accuracy with processing speed.

Retrieving more document chunks improves the accuracy of responses because the language model receives more context. However, retrieving too many chunks increases processing time and slows down system performance.

Trade-off Decision:

- Use a limited number of top relevant chunks
- Balance between response quality and speed
- Maintain efficient retrieval performance

This trade-off ensures that the system produces accurate responses without excessive delay.

7.3 Chunk Size vs Context Quality

Selecting the appropriate chunk size is another important challenge.

Large chunks contain more context, which improves response generation. However, large chunks may include irrelevant information and reduce retrieval precision.

Small chunks improve retrieval accuracy but may lose meaningful context.

Trade-off Decision:

- Use moderate chunk sizes
- Maintain balance between context retention and precision
- Improve retrieval relevance

This trade-off improves both accuracy and performance.

7.4 Retrieval Accuracy vs Storage Requirements

Storing a large number of embeddings increases retrieval accuracy because more detailed information is available. However, storing large embedding sets requires more storage space and increases memory usage.

Trade-off Decision:

- Store only the necessary document embeddings
- Optimize storage usage
- Maintain retrieval efficiency

This decision supports scalable system growth.

7.5 Response Quality vs Computational Cost

High-quality language models produce better responses but require more computational resources.

Using advanced models improves accuracy but increases processing cost and response time.

Trade-off Decision:

- Use efficient pre-trained models
- Balance performance and computational cost
- Optimize system efficiency

This trade-off improves system usability.

7.6 Automation vs Human Intervention

Automated systems provide fast responses but may produce incorrect answers in uncertain situations. Human intervention improves accuracy but increases response time.

Trade-off Decision:

- Use automated responses for common queries
- Trigger HITL for complex queries
- Maintain a balance between speed and reliability

This trade-off ensures high-quality responses.

7.7 Handling Large Documents

Processing large knowledge base documents introduces challenges related to chunking, storage, and retrieval efficiency.

Large documents increase processing time and memory usage.

Trade-off Decision:

- Split documents into manageable chunks
- Optimize indexing strategies
- Maintain efficient retrieval performance

This improves scalability and usability.

---

8. Testing Strategy

8.1 Overview of Testing Strategy

Testing is an important phase in the development of the RAG-Based Customer Support Assistant system. It ensures that the system performs correctly and produces accurate responses for user queries.

The testing strategy focuses on verifying the performance of individual components as well as the complete workflow. Proper testing helps identify errors, improve response accuracy, and ensure system reliability.

The testing process includes validating document processing, retrieval accuracy, response generation, and escalation logic.

8.2 Component-Level Testing

Component-level testing ensures that each module of the system functions correctly before integrating them into the complete workflow.

Each component is tested independently to verify its behavior.

Components Tested:

- Document Loading Module
- Text Chunking Module
- Embedding Generation Module
- Vector Storage Module
- Retrieval Module
- Language Model Module
- Workflow Execution Module

Testing each component individually helps detect errors early.

8.3 Retrieval Testing

Retrieval testing verifies whether the system retrieves relevant document chunks based on user queries.

The query embedding is compared with stored embeddings, and retrieved results are analyzed to ensure accuracy.

Testing Method:

- Provide sample queries
- Check retrieved document chunks
- Verify relevance of retrieved content

Example Queries Used:

- "How do I reset my password?"
- "How can I contact customer support?"
- "What should I do if my account is locked?"

Successful retrieval ensures meaningful response generation.

8.4 Response Generation Testing

Response generation testing ensures that the language model produces meaningful and accurate answers using retrieved context.

Testing Method:

- Provide test queries
- Verify generated responses
- Compare responses with knowledge base content

Example Test Cases:

Query:

"How can I update my account details?"

Expected Response:

The system should provide instructions related to updating account details based on the knowledge base.

This testing ensures response correctness.

8.5 Conditional Routing Testing

Conditional routing testing verifies that the system correctly decides whether to generate a response or escalate the query.

Testing Method:

- Provide queries with incomplete context
- Provide ambiguous queries
- Verify escalation triggers

Example Scenario:

Query:

"What is the refund policy for premium services?"

If the knowledge base does not contain refund details, escalation should be triggered.

This testing ensures correct decision-making.

8.6 HITL Testing

HITL testing ensures that human escalation works correctly.

Testing Method:

- Trigger a low-confidence scenario
- Forward query to human operator
- Verify human response integration

Expected Result:

The system should accept human responses and deliver them to users without interruption.

This ensures reliable human intervention.

8.7 End-to-End Workflow Testing

End-to-end testing verifies the complete workflow from user query to response delivery.

Testing Steps:

User submits query

System processes the query

Relevant chunks are retrieved

Response is generated

Output is delivered

Escalation occurs if necessary

This testing ensures the entire system functions smoothly.

---

9. Future Enhancements

9.1 Overview of Future Enhancements

Although the current RAG-Based Customer Support Assistant system provides accurate responses using document-based retrieval and language models, there are several opportunities to improve the system further.

Future enhancements focus on improving system intelligence, scalability, user interaction, and performance. These improvements will help transform the current prototype into a production-ready system.

9.2 Multi-Document Support

Currently, the system processes a single knowledge base document. In future versions, support for multiple documents can be added.

This enhancement would allow the system to retrieve information from multiple sources such as product manuals, service guides, and policy documents.

Benefits:

- Supports larger knowledge bases
- Improves information coverage
- Enables enterprise-level applications
- Multi-document support increases system scalability.

9.3 Feedback Learning System

A feedback-based learning system can be introduced to improve response quality over time.

Users or human operators can provide feedback on generated responses. This feedback can be stored and used to improve future responses.

Benefits:

- Improves response accuracy
- Reduces repeated errors

- Enables system self-improvement

Feedback learning enhances system intelligence.

9.4 Memory Integration

Memory integration allows the system to remember previous interactions with users.

This feature enables personalized responses based on conversation history.

Benefits:

- Improves conversational continuity
- Enhances user experience
- Supports multi-turn conversations

Memory integration improves interaction quality.

9.5 Web-Based User Interface

Currently, the system can be implemented using a Command Line Interface (CLI). In the future, a web-based interface can be developed to improve usability.

This enhancement allows users to interact with the system through a graphical interface.

Benefits:

- Improves user accessibility
- Enhances user experience
- Supports real-time interactions

Web interfaces make the system more user-friendly.

9.6 Real-Time Deployment

The system can be deployed on cloud platforms for real-time usage.

Cloud deployment enables the system to handle multiple users simultaneously and provide faster responses.

Possible Deployment Platforms:

- Cloud servers
- Web applications
- API-based services

Real-time deployment improves scalability.

9.7 Performance Optimization

Performance optimization techniques can be implemented to improve system speed and efficiency.

These improvements may include optimized retrieval strategies and faster embedding models.

Benefits:

- Reduces response time
- Improves processing efficiency
- Enhances overall performance

Optimization supports large-scale deployment.